AI-Powered Resume Analyzer and Career Copilot

1. Introduction

1.1. Project Overview

The AI-Powered Resume Analyzer and Career Copilot is an advanced, serverless Single Page Application (SPA) designed to revolutionize how job seekers optimize their career assets. Built using React.js (Vite), TypeScript, and Tailwind CSS for a highly responsive frontend, the application entirely bypasses traditional backend infrastructures (such as Node.js/Express.js) by natively embedding the Puter.js SDK.

As demonstrated in the core workflow architecture, all crucial operations—including user session management, secure document hosting, and high-performance Large Language Model (LLM) queries (GPT-4o/Claude 3.5 Sonnet)—are executed directly from the client side through Puter's secure cloud gateway, eliminating server maintenance costs and API key exposure risks.

1.2. Problem Statement

The contemporary job market is heavily governed by automated Applicant Tracking Systems (ATS), which automatically filter out an overwhelming majority of applicants based on rigid structural parameters, lack of matching keywords, or poor formatting before a human recruiter ever reviews the document. Traditional solutions are often hidden behind expensive paywalls or offer rudimentary keyword-matching algorithms without semantic awareness. Additionally, manual preparation of custom cover letters and targeted mock interview answers remains incredibly tedious and time-consuming for fresh graduates and job hunters.

1.3. Objectives

- **Eliminate Server Overhead:** Implement a fully functional, production-ready serverless environment using Puter.js to completely manage server logic and database storage on the client side.
- **Ensure Zero-Exposure Security:** Establish end-to-end encryption and tokenized requests for cloud AI services without exposing master API keys in front-end configurations.
- **Provide Context-Aware Scoring:** Utilize advanced semantic parsing via LLMs to generate real-time ATS match metrics spanning grammar, formatting, skill dense alignment, and professional tone.
- **Automate Career Asset Production:** Dynamically compile resume flaws and targeted job listings to output tailored cover letters and highly rigorous behavioral/technical mock interview questionnaires.

1.4. Scope of the Project

The operational boundary of this project is confined to a browser-executable environment optimized across all standard modern viewports. The system handles PDF processing, secure multi-version file storage, and context-aware chat streaming. Because it leverages Puter's unified zero-cost ecosystem, the system minimizes infrastructure limits while maximizing developer agility within a 15-week academic calendar.

2. Functional Requirements (FR)

Functional requirements specify the dynamic actions, inputs, processes, and outputs the software must handle.

- **FR-01 (Puter Authentication Handler):** The system must facilitate secure, passwordless, or credentialed user onboarding utilizing the native **puter.auth.signIn()** widget, changing global states to authenticated upon confirmation.
- **FR-02 (Persistent User Dashboard):** Upon successful authentication, the system must render an interactive user dashboard that tracks previous file metrics, performance graphs, and history logs.
- **FR-03 (Secure Cloud File Vault):** Users must be able to upload multi-page PDF resumes via a drag-and-drop box, prompting the system to execute a serverless upload using **puter.fs.write()** to isolated cloud buckets.
- **FR-04 (Asynchronous Document Rendering):** The application must process uploaded PDFs to display structural, background page previews instantly on the UI.
- **FR-05 (Job Description Extraction Field):** The system must provide a dedicated text entry layout capable of receiving raw, unformatted text strings representing target job advertisements or roles.
- **FR-06 (Semantic AI-ATS Processing Engine):** The system must merge extracted resume text strings and target job descriptions into a pre-engineered structural prompt, sending it via **puter.ai.chat()** to return a strict JSON payload containing specific metric ratings (0–100%) and bulleted improvements.
- **FR-07 (Automated Cover Letter Builder):** The system must offer an automated module to draft highly contextual, structured, and grammatically flawless cover letters tailored to the input criteria.
- **FR-08 (Targeted Interview Questionnaire Generator):** The system must formulate realistic technical and behavioral questions that map directly to the detected weaknesses in the parsed resume relative to the targeted job description.
- **FR-09 (Client-Side Document Export):** The application must feature client-side print/save capabilities allowing users to instantly download generated feedback reports, cover letters, and interview modules as perfectly formatted PDFs.

3. Non-Functional Requirements (NFR)

Non-functional requirements dictate the performance boundaries, quality traits, and constraints of the architecture.

- **NFR-01 (UI Latency & Speed):** Standard UI routing, panel transitions, and local state updates must execute within ≤ 1 second. High-capacity cloud AI queries must render structured visualizations within 5 to 8 seconds, depending on remote model server availability.
- **NFR-02 (Zero-Config Security Architecture):** The frontend code must strictly avoid embedding hardcoded environment files **(.env)** or sensitive API keys for OpenAI/Anthropic services, delegating all verification to Puter's infrastructure runtime.
- **NFR-03 (State Management Predictability):** Global system variables (e.g., authentication status, active file data, active report streams) must be handled by **Zustand** to maintain unidirectional, type-safe data flow across React re-renders.
- **NFR-04 (Fluid Responsive Layouts):** The graphical interface must utilize utility-first styling with **Tailwind CSS**, adapting to varying display sizes from 320px mobile displays up to standard 4K desktop screens.
- **NFR-05 (Infrastructure Availability):** The system must operate with zero hosting downtime by using Puter's globally distributed cloud CDN edge networks.

4. User Roles

The platform identifies two specific operational user roles interacting with the system.

4.1. Job Seeker / Candidate (Primary End-User)

The Job Seeker represents the standard operational user profile for whom the system’s core functionalities are explicitly built. They interact with the front-end graphical interfaces to upload assets and manage their personal career growth pathways.

- **System Onboarding & Session Control:** Responsible for initiating accounts and securing validation workflows via the Puter OAuth interface.
- **Document Vault Management:** Exercises total control over personal document repositories, including uploading multi-page PDF resumes, assigning specific version names, tracking historical records, and purging redundant profiles from cloud storage.
- **Job Targeting Configuration:** Copies and pastes context-heavy job postings into the data processor to establish the benchmarking criteria for evaluations.
- **Triggering AI Assessments:** Authorizes the application to compile their resume content against job configurations, executing semantic analyses via **puter.ai.chat()**.
- **Asset Retrieval & Local Archiving:** Evaluates real-time, color-coded score metrics, requests the synthesis of secondary data assets (Cover Letters/Interview Sheets), and exports finalized elements as clean local PDFs.

4.2. Puter Ecosystem Administrator (System Component Actor)

The System Administrator is a passive/infrastructure-level role that manages global cloud operations, network traffic, and underlying API resources. Instead of custom server-side staff, this role is handled directly by the Puter cloud backend.

- **Cloud Infrastructure & API Gatekeeping:** Monitors global application status, handles operational proxy routines, and manages cloud routing protocols for client-side API requests.
- **Storage Allocation & Data Custody:** Governs cloud disk spaces, maintains NoSQL Key-Value infrastructure integrity, and protects user database files from external data leaks.
- **LLM Request Orchestration:** Manages token allocations, rate-limiting rules, and secure proxy bridges to upstream model APIs (GPT-4o/Claude 3.5 Sonnet).

5. Use Cases

Use Case 1: Client-Side User Login

- **Actor:** Job Seeker
- **Action:** The user clicks the "Login with Puter" action button on the landing index page.
- **System Response:** The application invokes the Puter SDK framework, triggering a secure modal popup window. Upon the user granting data permissions, the system captures a signed session token, updates the global state to **isLoggedIn: true**, and grants access to the Personal Analytics Dashboard.

Use Case 2: Resume Upload to Puter FS

- **Actor:** Job Seeker
- **Action:** The user selects or drops a raw PDF file onto the designated drag-and-drop dashboard panel.
- **System Response:** The React engine intercepts the file buffer stream, fires a **puter.fs.write('/resumes/' + unique_id + '.pdf', fileData)** routine to push the document directly to cloud storage, appends the metadata parameters into the NoSQL repository, and alerts the user with a localized success notification.

Use Case 3: Prompt Engineering & AI Audit Execution

- **Actor:** Job Seeker
- **Action:** The user highlights an active resume from their vault, enters a target job description, and triggers the "Run Diagnostic" switch.
- **System Response:** The system runs a data-mapping routine that reads the file text string alongside the job post text string, structures a system-wide optimization prompt, and dispatches it via a single asynchronous **puter.ai.chat()** function call. Once the JSON payload returns, the front-end parses it to dynamically update numerical graphs and actionable text sections on the screen.